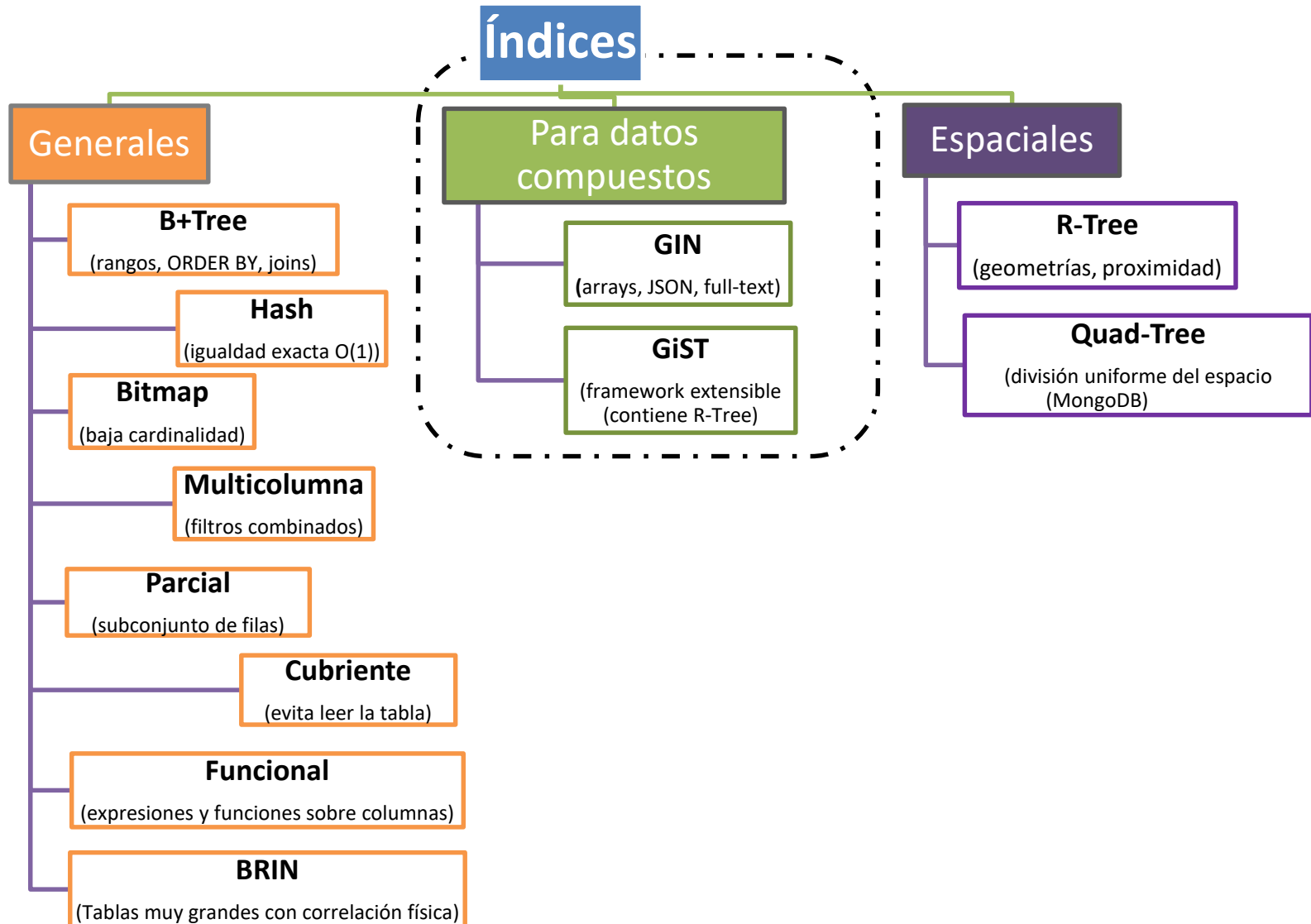




Índices en Base de Datos



Índices en Base de Datos

Índices para datos compuestos

Índices en Base de Datos

Índices GIN (Generalized Inverted Index)

Índices en Base de Datos

Índices GIN

¿Qué son los índices GIN?

- Usando la analogía son como el **índice de un libro**.
- No está ordenado por página sino por palabra
“Está palabra aparece en las páginas 12,42 y 89”
- **Indexa cada elemento interno** de una colección o documento.

Índices en Base de Datos

Índices GIN

GIN es el índice perfecto para columnas que contienen **múltiples valores o documentos**, como etiquetas, listas o textos largos.

PostgreSQL lo usa ampliamente en **búsqueda semántica, análisis de texto y estructuras JSON**.

GIN pregunta *"¿está este valor en el conjunto?"*
— busca exacta.

Índices en Base de Datos

Índices GIN

Construye un índice invertido:

- para cada elemento (palabra, tag, clave JSON)
- guarda la lista de filas donde aparece.

Una búsqueda va directo a esa lista sin recorrer la tabla.

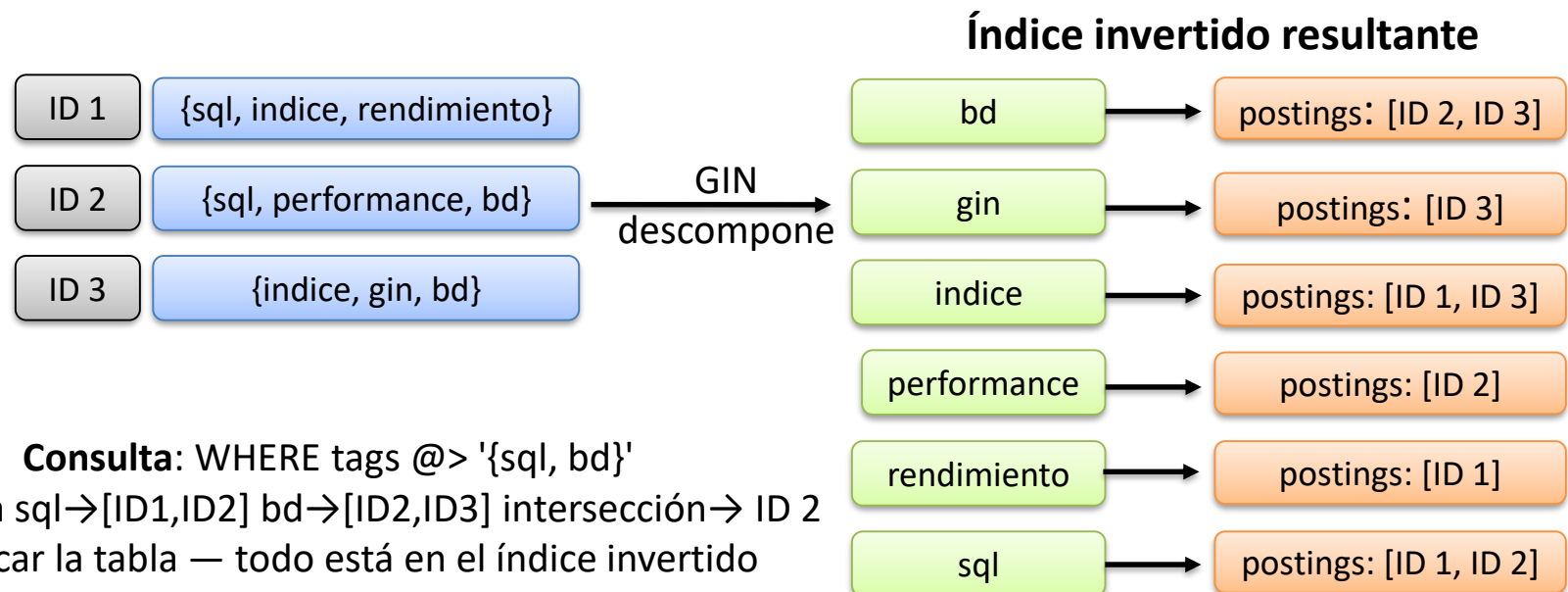
Índices en Base de Datos

Índices GIN

¿Cómo descompone GIN?

Construye un índice invertido

para cada elemento (palabra, tag, clave JSON) guarda la lista de filas donde aparece. Una búsqueda va directo a esa lista sin recorrer la tabla.



Índices en Base de Datos

Índices GIN

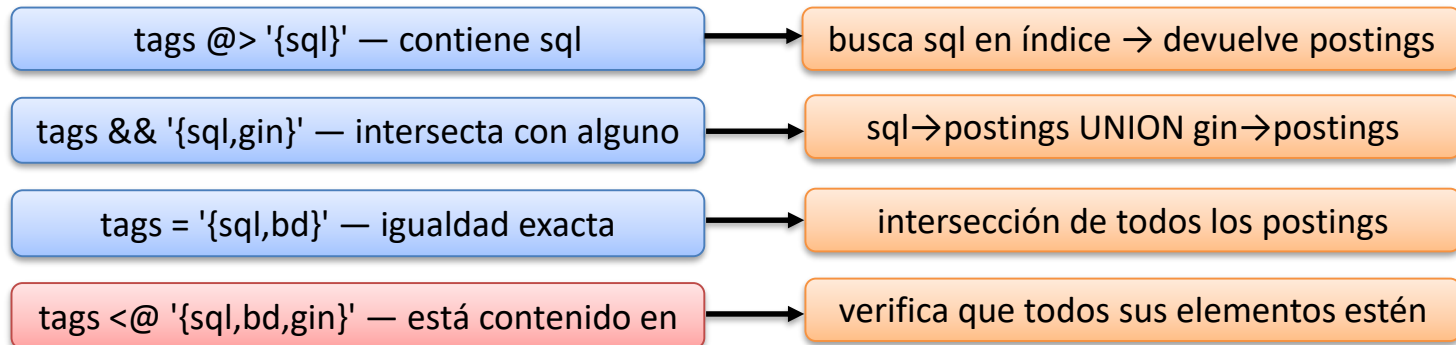
GIN con arrays

soporta cuatro operadores clave.

El más común es `@>` (contiene).

La intersección y unión de posting lists se hace a nivel de índice (rapidísimo).

Operadores disponible



- GIN NO soporta: `<`, `>`, BETWEEN, ORDER BY — no tiene noción de orden
- INSERT y UPDATE son más lentos porque reconstruye el índice invertido
- Solución: `fastupdate=on` acumula cambios en lotes para reducir el costo

Índices en Base de Datos

Índices GIN

GIN con arrays

Crear índice GIN sobre array de tags

```
CREATE INDEX idx_gin_tags ON articulos USING GIN (tags);
```

Operadores disponibles

```
SELECT * FROM articulos WHERE tags @> '{sql}';           (contiene)
```

```
SELECT * FROM articulos WHERE tags && '{sql, gin}';       (intersecta)
```

```
SELECT * FROM articulos WHERE tags = '{sql, bd}';        (igualdad exacta)
```

```
SELECT * FROM articulos WHERE tags <@ '{sql,bd,gin}';     (contenido en)
```

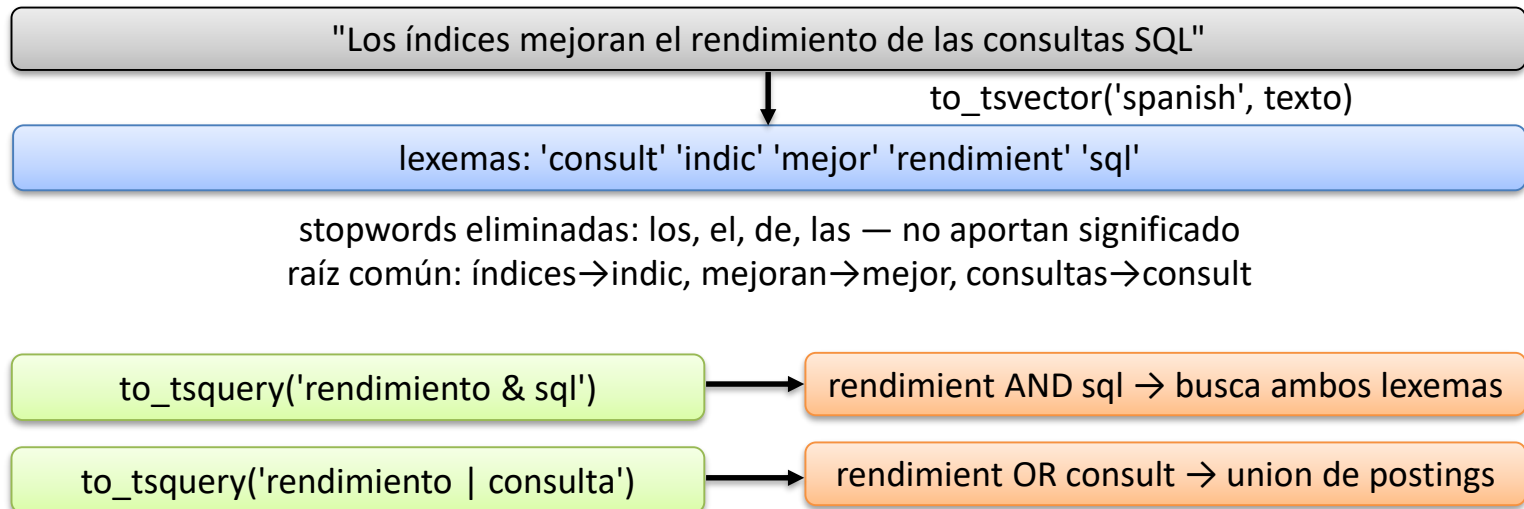
Índices en Base de Datos

Índices GIN

GIN con full-text

- Full-text search convierte el texto a **lexemas** (raíces normalizadas sin stopwords).
- GIN indexa cada lexema.
- La búsqueda combina posting lists con AND/OR según el tsquery.

GIN con full-text search — tsvector y tsquery



GIN guarda cada lexema → lista de documentos que lo contienen

Índices en Base de Datos

Índices GIN

GIN con full-text search

```
CREATE INDEX idx_gin_fts ON articulos  
    USING GIN (to_tsvector('spanish', cuerpo));
```

Búsqueda con operadores booleanos

```
SELECT * FROM articulos  
WHERE to_tsvector('spanish', cuerpo) @@ to_tsquery('rendimiento & sql');
```

```
SELECT * FROM articulos  
WHERE to_tsvector('spanish', cuerpo) @@ to_tsquery('indice | performance');
```

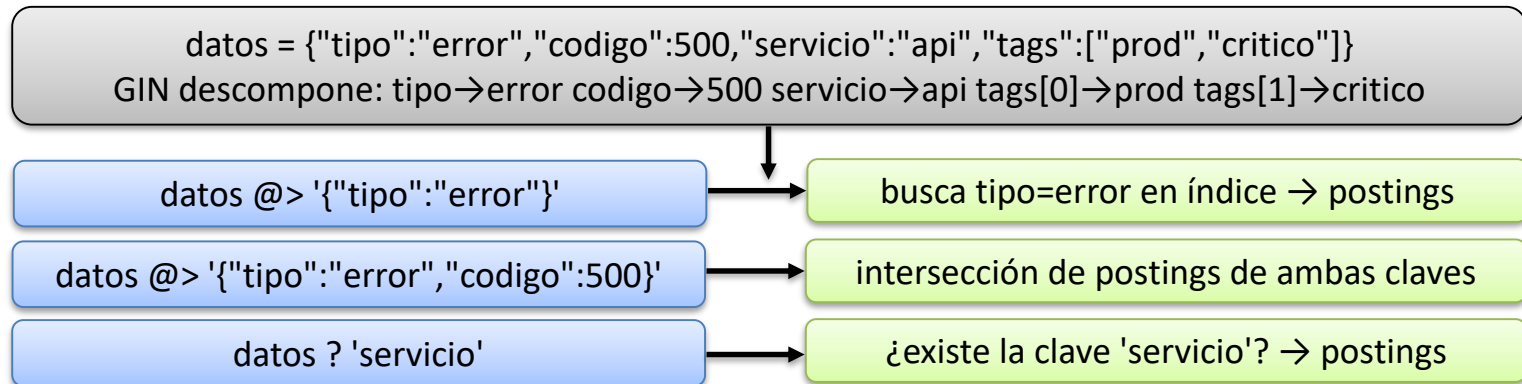
Índices en Base de Datos

Índices GIN

GIN con JSONB

- descompone cada documento en pares **clave-valor**.
- Dos opciones:
 - jsonb_ops (indexa todo)
 - jsonb_path_ops (solo valores, más eficiente para @>).

GIN con JSONB — índice sobre documentos JSON



jsonb_ops (default): indexa claves, valores y estructuras (mas pesado)
jsonb_path_ops: solo indexa valores — más compacto, solo soporta @>
 Usar jsonb_path_ops cuando se necesita @> — ocupa 3x menos espacio

Índices en Base de Datos

Índices GIN

GIN con JSONB (jsonb_ops — indexa todo)

```
CREATE INDEX idx_gin_json ON eventos  
    USING GIN (datos [jsonb_ops]);
```

GIN con JSONB (jsonb_path_ops — solo @>, más compacto)

```
CREATE INDEX idx_gin_json_path ON eventos  
    USING GIN (datos jsonb_path_ops);
```

```
SELECT * FROM eventos WHERE datos @> '{"tipo":"error"}';
```

```
SELECT * FROM eventos WHERE datos @> '{"tipo":"error","codigo":500}';
```

```
SELECT * FROM eventos WHERE datos ? 'servicio'; (solo con jsonb_ops)
```

Índices en Base de Datos

Índices GiST (Generalized Search Tree)

Índices en Base de Datos

Índices GiST

Si **GIN** es el índice del libro, **GiST** es un **mapa jerárquico**:

- organiza los datos por proximidad o solapamiento
- no por valor exacto.
- Es un framework extensible — la misma estructura sirve para geometrías, rangos de fechas, texto, vectores, documento, y más.

Índices en Base de Datos

Índices GiST

¿Cómo funciona GiST?

Cada nodo del árbol almacena un **bounding box**¹ (la región mínima que contiene todos sus hijos).

Al buscar, descarta ramas enteras cuyo bbox no se solapa con la consulta.

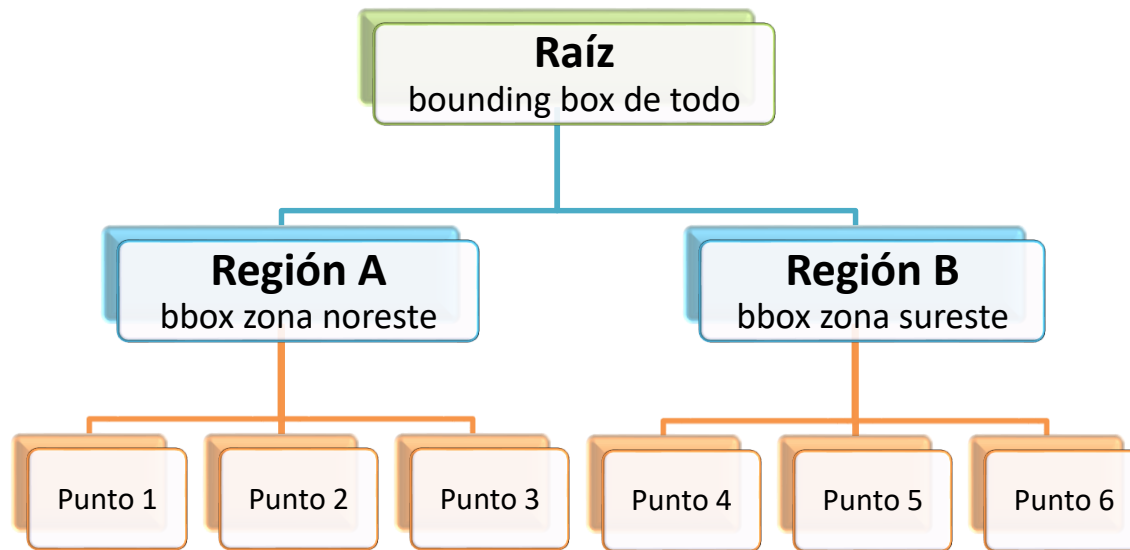
¹ Rectángulo mínimo que encierra un objeto geométrico.

Índices en Base de Datos

Índices GiST

GiST organiza datos en regiones jerárquicas

(cada nodo es un bounding box)



Consulta: ¿qué puntos están dentro de mi radio?

GiST compara el radio contra el bbox de cada región

Si el bbox no se solapa con el radio → descarta toda la rama
sin revisar ninguno de los puntos dentro de ella

¹ Rectángulo mínimo que encierra un objeto geométrico.

Índices en Base de Datos

Índices GiST

Busqueda por radio

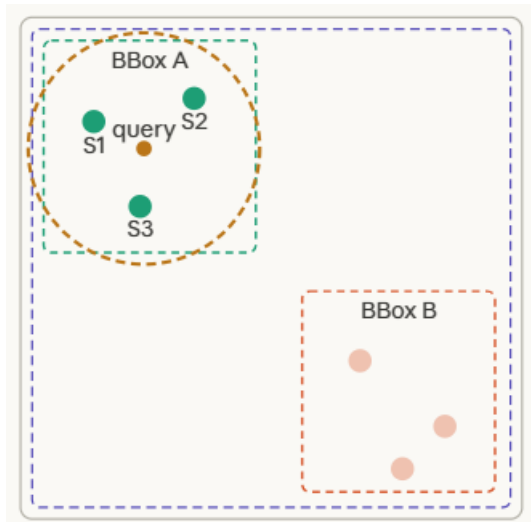
Para datos espaciales, GiST organiza los puntos en bounding boxes jerárquicos.

Una búsqueda por radio compara el círculo contra los bboxes:

si no se tocan, descarta toda la rama sin revisar ningún punto dentro de ella.

Búsqueda por radio — ST_DWithin

WHERE ST_DWithin(ubicacion, punto_consulta, radio)



Solo revisó 3 puntos de 6 — descartó BBox B entero sin leer ninguno de sus puntos
Con millones de puntos la diferencia es de segundos vs milisegundos

Índices en Base de Datos

Índices GiST

Busqueda por área

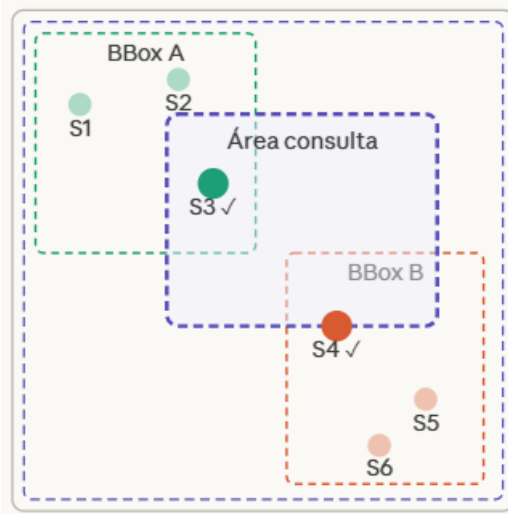
Para búsqueda por área, el polígono de consulta se compara contra cada BBox.

Si se solapan, el motor baja a revisar los puntos individuales de esa rama.

Si no se tocan, descarta la rama completa.

Búsqueda por área — ST_Intersects

WHERE ST_Intersects(ubicacion, poligono_consulta)



Navegación del árbol

BBox A se solapa con área → entrar

S1 ✗ fuera S2 ✗ fuera
S3 ✓ dentro del área



BBox B se solapa con área → entrar

S4 ✓ dentro S5 ✗ fuera
S6 ✗ fuera del área



Resultado: S3 y S4

revisó 6 puntos, descartó 4

Ambos BBoxes se solapan → ambos se inspeccionan

GiST hace dos pasadas: **primero** filtra por bbox (rápido), **luego** verifica exactamente (preciso)
Con millones de puntos y BBoxes bien organizados, descarta el 99% sin leerlos

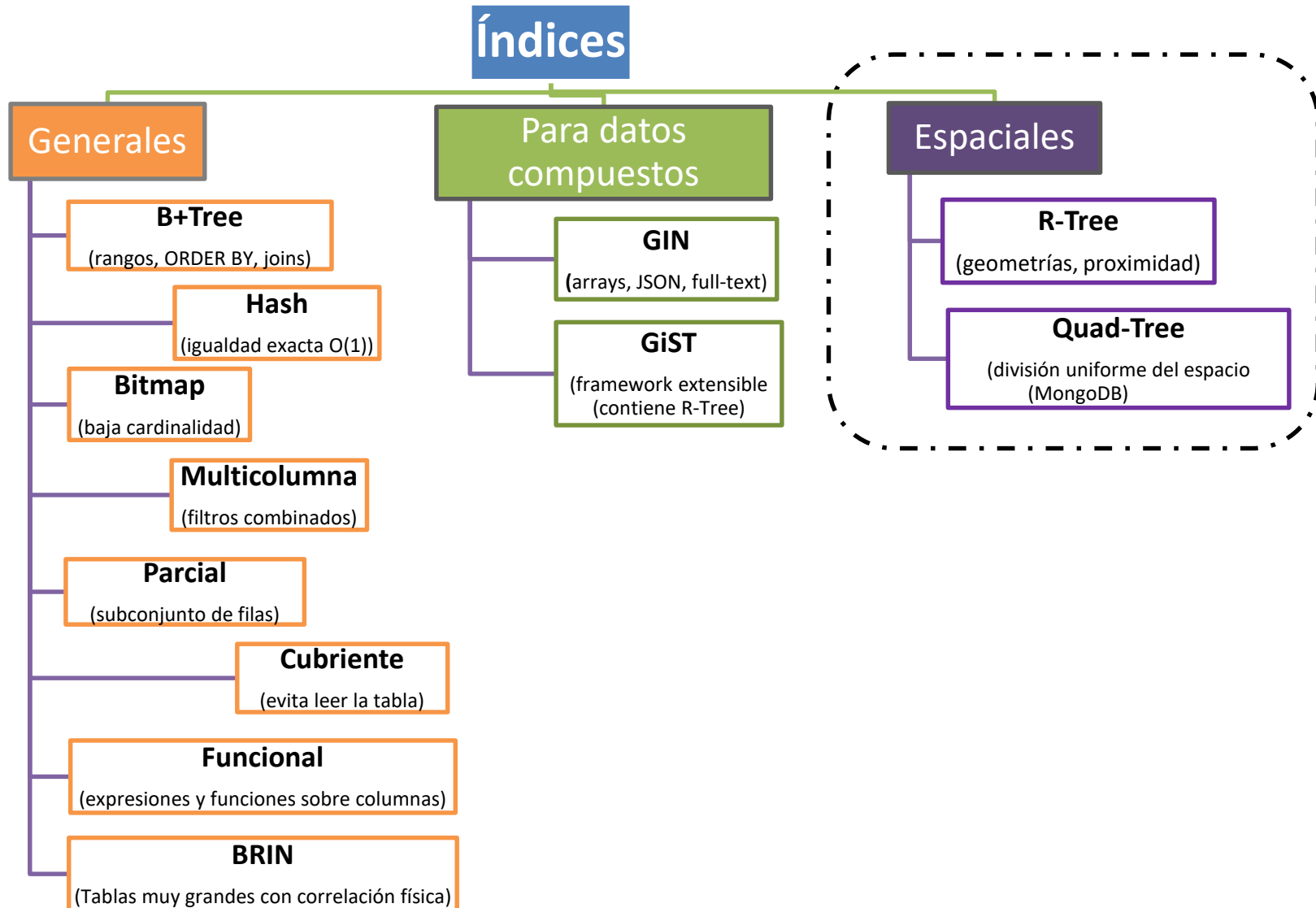
Índices en Base de Datos

Índices GiST

GiST con ST_Intersects (búsqueda por área - PostgreSQL)

```
CREATE TABLE IF NOT EXISTS sucursales (  
  id SERIAL PRIMARY KEY,  
  nombre VARCHAR(100),  
  ubicacion GEOMETRY(Point, 4326) (SRID 4326 = coordenadas geográficas WGS84)  
);  
  
CREATE INDEX idx_gist_geo ON sucursales  
USING GIST (ubicacion);  
  
SELECT nombre FROM sucursales  
WHERE ST_Intersects(  
  ubicacion,  
  ST_GeomFromText('POLYGON((-68.2 -34.5,  
                                -68.0 -34.5,  
                                -68.0 -34.7,  
                                -68.2 -34.7,  
                                -68.2 -34.5 ))',  
                                4326)  
);
```

Índices en Base de Datos



Índices en Base de Datos

Índices espaciales

Índices en Base de Datos

Índices R-Tree

Índices en Base de Datos

Índices R-Tree

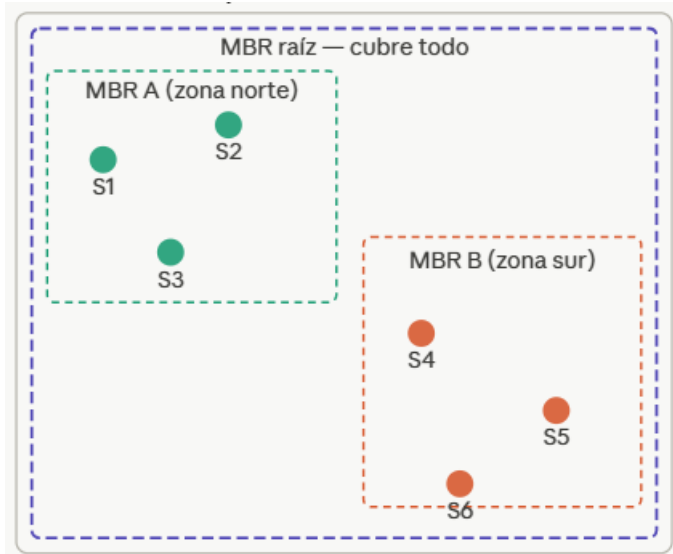
- Los índices R-Tree son los más usados en bases de datos espaciales.
- Representa la estructura que está **detrás de GiST** en PostgreSQL y de los índices espaciales en MySQL/MariaDB.
- La idea es envolver grupos de objetos en **rectángulos mínimos** (Minimum Bounding Rectangle, **MBR**) que se van anidando.

Índices en Base de Datos

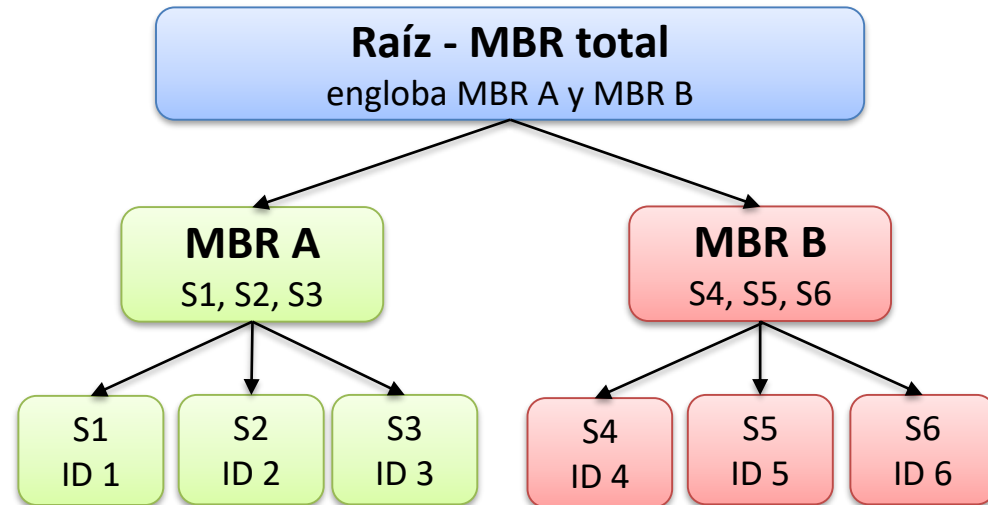
Índices R-Tree

Estructura MBR

Espacio 2D - Sucursales



Árbol R-Tree interno



Cada nodo interno es un MBR que engloba a todos sus hijos
Los MBRs pueden solaparse entre sí

Índices en Base de Datos

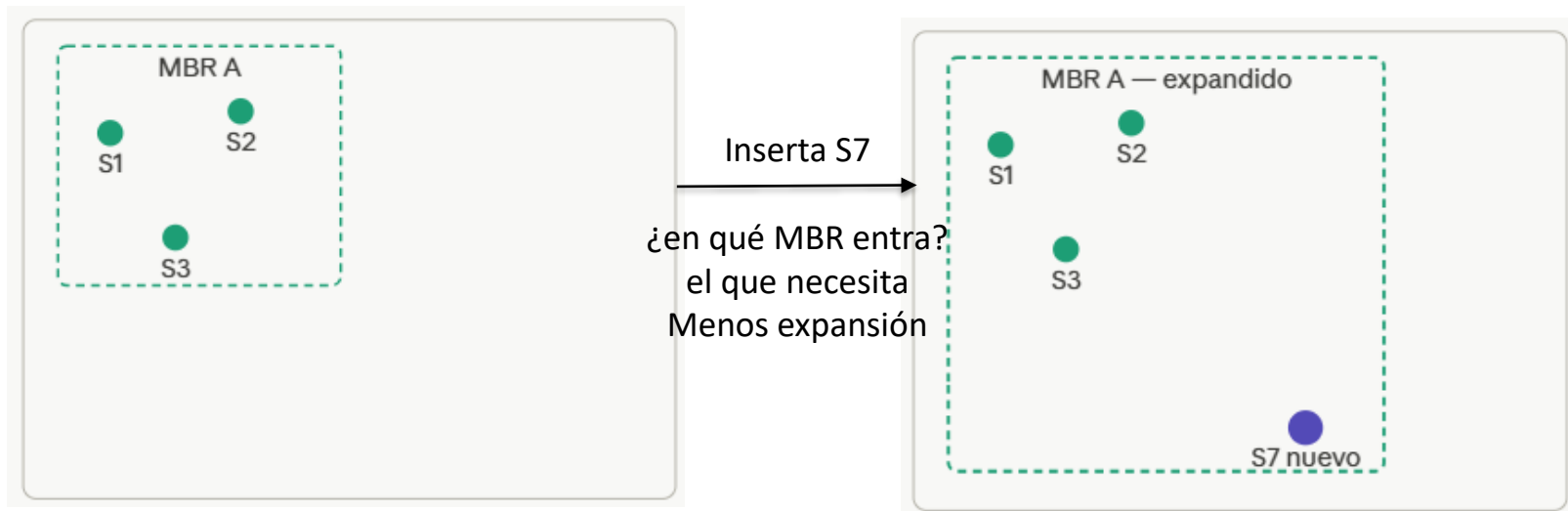
Índices R-Tree

Inserción (el MBR se agranda si es necesario)

Al insertar un nuevo punto, elige el MBR que necesita menos área adicional para contenerlo. Si el nodo se llena, se divide (Split) y el árbol puede crecer en altura.

Antes - MBR A cubre S1, S2, S3

Después - MBR A se expande para cubrir S7



El algoritmo elige el MBR cuya área crece menos al incluir el nuevo punto
Si el nodo se llena (supera el máximo de entradas) → se divide en dos nodos: Split
El Split propaga hacia arriba y puede crecer el árbol en altura

Índices en Base de Datos

Índices R-Tree

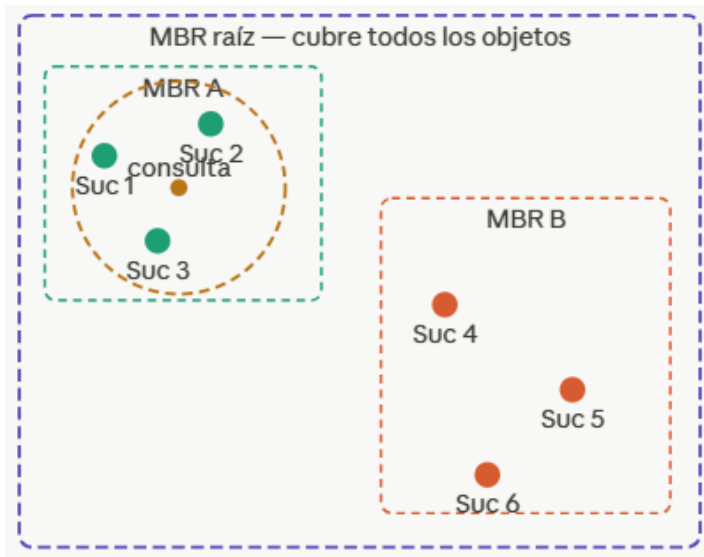
Búsqueda por radio

Compara el círculo contra cada MBR.

Si no se solapan, descarta toda la rama. Solo entra en las ramas donde el MBR toca el radio - ahorra la inmensa mayoría de las comparaciones.

Búsqueda por radio - ST_DWithin

WHERE ST_DWithin(ubicacion, punto_consulta, radio)



Pasos de navegación

1. Raíz: ¿radio $\cap$ MBR raíz?

SÍ $\rightarrow$ entrar al árbol

2. Raíz: ¿radio $\cap$ MBR A?

SÍ $\rightarrow$ revisar S1, S2, S3

3. Raíz: ¿radio $\cap$ MBR B?

No $\rightarrow$ descartar S4, S5, S6

4. Verificar S1, S2 S3

S1 ✓ S2 ✓ S3 ✓
dentro del radio

Solo revisó 3 puntos de 6 — descartó MBR B entero sin leer sus puntos

Índices en Base de Datos

Índices R-Tree

Búsqueda por radio

PostgreSQL + PostGIS: R-Tree vía GiST

CREATE INDEX idx_rtree **ON** sucursales **USING** GIST (ubicacion);

Crea un índice espacial sobre la columna ubicacion de la tabla sucursales.

Radio de búsqueda (10km)

SELECT nombre **FROM** sucursales

WHERE ST_DWithin(ubicacion, ST_MakePoint(-68.13, -34.60)::geography, 10000);

*Busca todas las sucursales cuya ubicación esté dentro de **10.000 metros (10 km)** del punto (-68.13, -34.60) (coordenadas de lat/long).*

Vecino más cercano (K-NN - k-nearest neighbors)

SELECT nombre **FROM** sucursales **ORDER BY** ubicacion <-> ST_MakePoint(-68.13, -34.60)::geography

LIMIT 5;

Ordena todas las sucursales por distancia al punto dado y devuelve las 5 más cercanas.

Índices en Base de Datos

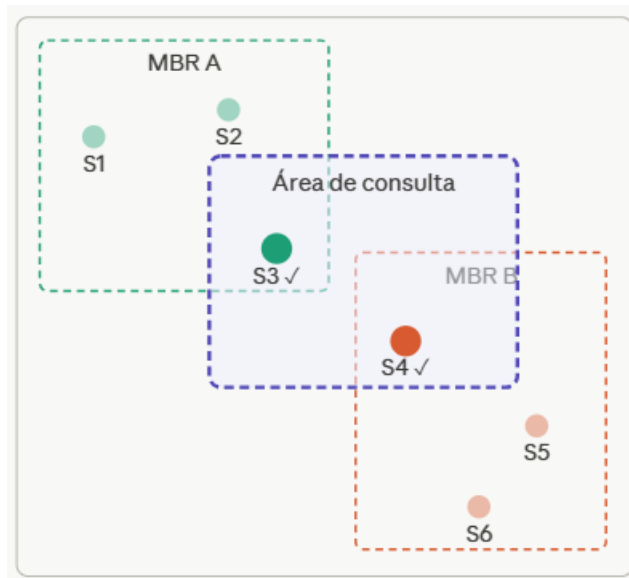
Índices R-Tree

Búsqueda por área

Igual que por radio pero con un polígono.

El R-Tree descarta MBRs que no se solapan con el área y solo verifica los puntos de los MBRs que sí podrían contener resultados.

Búsqueda por área - ST_Intersects / ST_Contains



Qué revisa el motor

MBR A se solapa con área → entrar

S1 ✗ S2 ✗ S3 ✓ dentro del área

MBR B se solapa con área → entrar

S4 ✓ S5 ✗ S6 ✗ fuera del área

Resultado: S3 y S4

Revisó 6 puntos descartó 4

Con millones de puntos y MBRs bien organizados, descarta 99% de los datos sin leerlos

Índices en Base de Datos

Índices R-Tree

Búsqueda por área

MySQL: R-Tree nativo con SPATIAL INDEX

CREATE SPATIAL INDEX idx_rtree **ON** sucursales (ubicacion);

*Crea un índice espacial tipo **R-Tree** sobre la columna ubicacion.*

Búsqueda por polígono (bounding box)

SELECT nombre **FROM** sucursales

WHERE ST_Intersects(
 ubicacion,
 ST_GeomFromText('POLYGON((...))'));

Busca todas las sucursales cuya ubicación (POINT) esté dentro o toque el polígono definido.

MariaDB: misma sintaxis que MySQL

CREATE INDEX idx_geo **ON** sucursales (punto) **USING** SPATIAL;

Índices en Base de Datos

Índices R-Tree

- En PostgreSQL el R-Tree vive dentro de GiST (no existe CREATE INDEX USING RTREE).
- En MySQL/MariaDB existe SPATIAL INDEX que usa R-Tree directamente.
- Todos los operadores espaciales de PostGIS aprovechan el R-Tree.

R-Tree en producción - PostgreSQL y MySQL

PostgreSQL - vía GiSTR-Tree

implementado dentro del framework GiST con PostGIS

USING GIST → usa R-Tree internamente
Soporta 2D y 3D, geografía real
y geometría proyectada

MySQL / MariaDB — R-Tree nativo

SPATIAL INDEX usa R-Tree directamente sin
framework intermedio
USING SPATIAL → R-Tree directo Solo 2D (más simple
pero más limitado que PostGIS)

Operadores espaciales con R-Tree

ST_DWithin → radio

ST_Intersects → toca

ST_Contains → contiene

ST_Within → está dentro

ST_Overlaps → se solapa

<-> → vecino más cercano

Todos estos operadores aprovechan el R-Tree para descartar MBRs antes de calcular la geometría real
Primero filtra por MBR (rápido), luego verifica exactamente (preciso)

Índices en Base de Datos

Índices Quad-Tree

Índices en Base de Datos

Índices Quad-Tree

Divide el espacio en **cuadrantes** fijos

- A diferencia del R-Tree que agrupa objetos en rectángulos variable
- divide el espacio en **4 cuadrantes iguales** recursivamente (Es como hacer zoom en google maps).

Índices en Base de Datos

Índices Quad-Tree

- Parte de cero: el espacio completo es el nodo raíz.
- Cuando una celda tiene más puntos que el umbral definido, se divide exactamente en 4 cuadrantes iguales.
- La división es siempre por el centro geométrico.

Índices en Base de Datos

Índices Quad-Tree

Nivel 0 – espacio completo

Parte de cero: el espacio completo es el nodo raíz.

Cuando una celda tiene más puntos que el umbral definido, se divide exactamente en 4 cuadrantes iguales.

La división es siempre por el centro geométrico.

Espacio completo - sin dividir



Árbol - nivel 0

Raíz

todos los puntos: S1..S7

7 puntos en un solo nodo.

Si el umbral es 4 puntos
por nodo → se subdivide
en 4 cuadrantes iguales.

Sin organización → búsqueda por radio compara todos los puntos uno por uno — $O(n)$

Índices en Base de Datos

Índices Quad-Tree

Nivel 1 – 4 cuadrantes

El espacio se divide con una cruz fija en el centro.

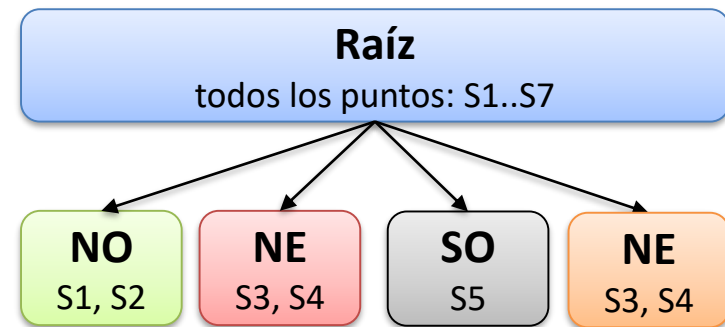
NO recibe S1 y S2 - **NE** recibe S3 y S4 - **SO** recibe S5 - **SE** recibe S6 y S7.

Los límites son siempre los mismos.

Nivel 1 — división en 4 cuadrantes fijos



Árbol - nivel 1



Sin organización → búsqueda por radio compara todos los puntos uno por uno — $O(n)$

Índices en Base de Datos

Índices Quad-Tree

Nivel 2 – NE se subdivide

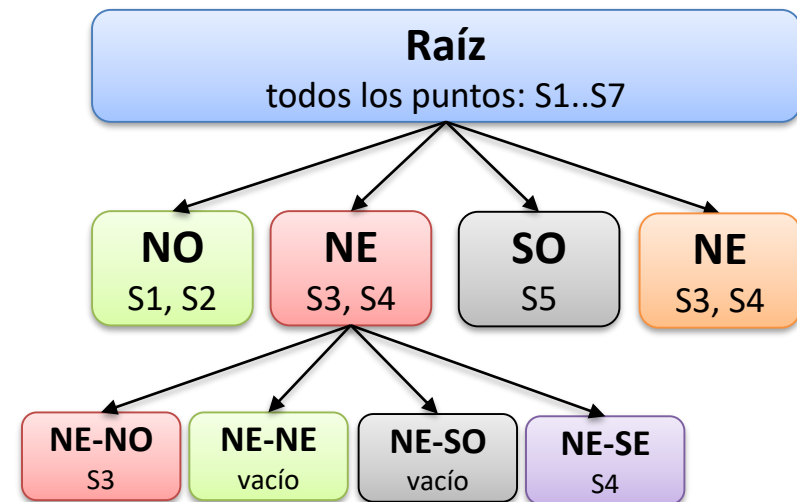
NE tiene 2 puntos (S3 y S4).

- Con umbral de 1 punto por nodo.
 - NE se subdivide en 4 sub-cuadrantes iguales.
 - S3 cae en NE-NO y S4 en NE-SE.
 - NE-NE y NE-SO quedan vacíos pero existen en el árbol.

Nivel 2 — cuadrante NE se subdivide



Árbol - nivel 2



NE-NE y NE-SO quedan vacíos — el árbol los mantiene aunque no tengan datos. Ese es el concepto del Quad-Tree.

Índices en Base de Datos

Índices Quad-Tree

Búsqueda por radio

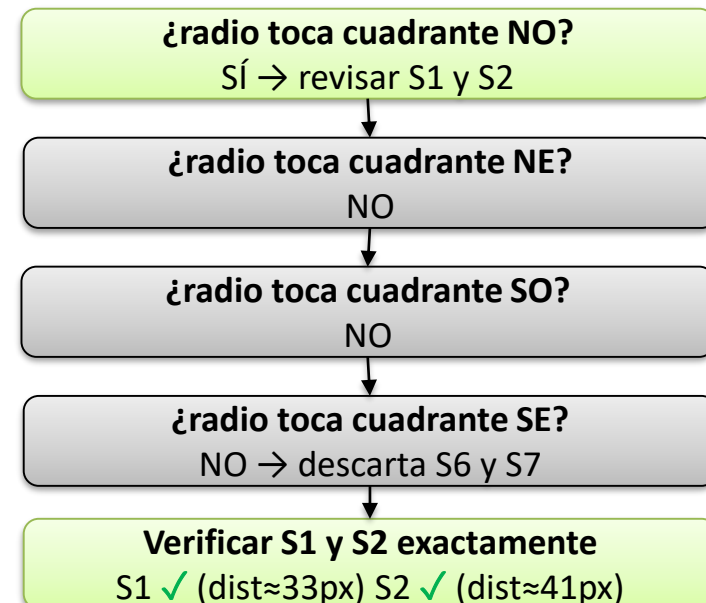
El radio queda completamente dentro del cuadrante NO.

- Su borde derecho llega a $x=145$, que es 15px antes de la frontera con NE ($x=160$).
- Su borde inferior llega a $y=165$, que es 5px antes de SO ($y=170$).
- El motor descarta NE, SO y SE sin revisar ningún punto.

Búsqueda por radio - solo toca NO



Pasos de la navegación



Índices en Base de Datos

Índices Quad-Tree

¿Dónde se usa el Quad-Tree?

Susa principalmente en MongoDB (índice 2d - videojuegos y mapas simples).
Geografía real en producción R-Tree (vía GiST (PostgreSQL)) elección estándar.

MongoDB - índice 2d

coordenadas planas (no esféricas)

Videojuegos

detección de colisiones en 2D

Mapas y GIS simples

Tiles de mapa (tamaño fijo), zoom levels

Imágenes y gráficos

compresión, detección de regiones

Cuándo elegir R-Tree vs Quad-Tree

R-Tree

datos geográficos reales (PostGIS)
datos distribuidos irregularmente
polígonos y geometrías complejas
producción con millones de puntos
consultas de vecino más cercano

Quad-Tree

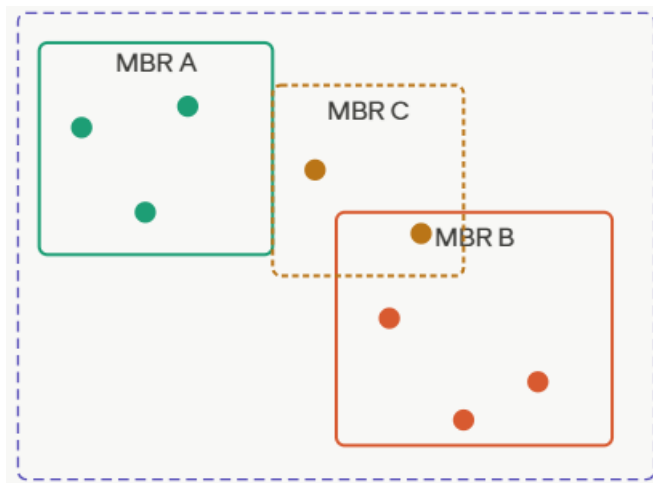
datos uniformemente distribuidos
espacio 2D plano (no esférico)
videojuegos y simulaciones
cuando los límites fijos son útiles
MongoDB con coordenadas 2d

Índices en Base de Datos

R-Tree vs Quad-Tree

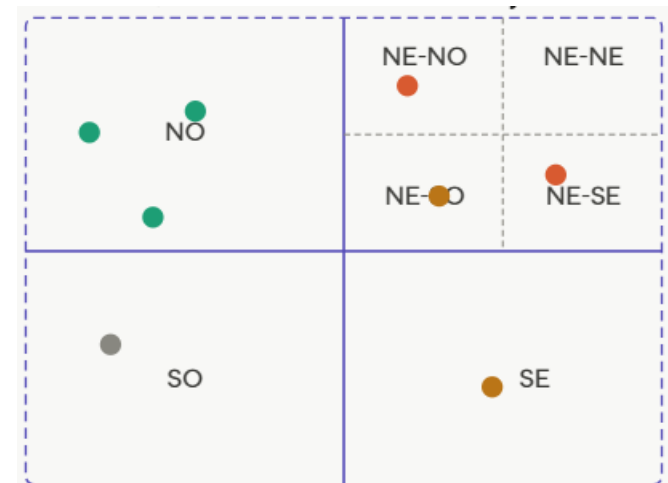
R-Tree vs Quad-Tree (la diferencia clave)

R-Tree - MBR ajustado a los datos



Los MBRs se ajustan a donde están los datos
Pueden solaparse entre sí - consulta puede entrar a varios
Eficiente con datos agrupados naturalmente

Quad-Tree – cuadrantes fijos



Los cuadrantes son siempre iguales y no se solapan
Zonas vacías igual existen en el árbol
Predecible y simple de implementar

Índices en Base de Datos

Bibliografía:

Bases de datos generales (cubren B+Tree, Hash, Bitmap, Multicolumna)

- **Ramakrishnan, R. & Gehrke, J.** *Database Management Systems* (3ra ed.) McGraw-Hill, 2003 → Capítulos 8 (almacenamiento), 9 (índices B+Tree y Hash), 10 (optimización de consultas) → El libro más usado en cursos universitarios para estos temas.
- **García-Molina, H., Ullman, J. & Widom, J.** *Database Systems: The Complete Book* (2da ed.) Pearson, 2008 → Capítulos 13 y 14 (índices, estructuras de almacenamiento) → Más teórico y formal que los anteriores.

Índices en Base de Datos

Bibliografía

Optimización y rendimiento (cubren Parcial, Cubriente, Funcional, BRIN)

- **Winand, M.** *SQL Performance Explained* Autoeditado, 2012 → Libro completo dedicado a índices en SQL. Cubre B+Tree, Multicolumna, Parcial, Cubriente y Funcional con ejemplos reales en PostgreSQL, MySQL y Oracle. → Disponible online gratis en **use-the-index-luke.com**.
- **Schwartz, B., Zaitsev, P. & Tkachenko, V.** *High Performance MySQL* (4ta ed.) O'Reilly, 2022 → Capítulo 8 (indexación y optimización de consultas) → Cubre índices Bitmap, Multicolumna, Cubriente y BRIN desde una perspectiva de producción.

Índices en Base de Datos

Bibliografía

Índices espaciales (R-Tree, Quad-Tree, GiST)

- **Rigaux, P., Scholl, M. & Voisard, A.** *Spatial Databases: With Application to GIS* Morgan Kaufmann, 2002
- **Guttman, A.** *R-Trees: A Dynamic Index Structure for Spatial Searching* ACM SIGMOD, 1984

GIN y GiST (índices para datos compuestos)

- **Stonebraker, M. et al.** *Generalized Inverted Indexes for Log Analysis* CIDR, 2009 → Base conceptual de GIN y los índices invertidos en bases de datos.

Base de Datos Avanzada

